Questions on March 30, 2026: Pointer

Tong Yi

Contents

1 Address of a Variable **2**

2 Why Use a Pointer? **2**

3 What are Pointers? **3**
3.1 Dereferencing Pointers 3
3.2 Safety: Null and Uninitialized Pointers 3

4 What is the Difference Between `int* p` and `int *p`? **4**

5 What is a String Pointer? **4**

6 What are the Common Uses for Two Pointers (Answer to the Lecture Slip)? **4**

7 Integer Array Passes as a Pointer to Integer **6**

8 What do `*variable` and `&variable` mean? **7**

9 Why use `nullptr` instead of `NULL` or 0? **7**

10 How to Delete a Pointer? **7**

11 When to Use * (Pass by Pointer) and When to Use & (Pass by Reference) ? **8**

12 When to Use Const for a Function **12**

13 Did I use the right steps to get the string pointer? **13**

14 I still don't fully understand pointers **13**

15 Will Monte Carlo pi simulation be on the final? **14**

16 Not Enough Time for the Lecture Slip Question **14**

17 I Need the Code Posted **14**

1 Address of a Variable

The address of a variable can be found using the address-of operator (`&`). When placed before a variable name, it returns the memory address assigned to that variable by the system.

Note that memory is managed by the operating system. Because different applications run at different times, the actual addresses allocated to your program may change each time you run it. However, the **relative** spacing between variables (such as elements in an array) remains consistent.

```cpp
#include <iostream>

//sample input/output:
//the address of a is 0x7ff7b6a50558
//0x7ff7b6a50560 is the same as 0x7ff7b6a50560
//0x7ff7b6a50564 is the same as 0x7ff7b6a50564
//0x7ff7b6a50568 is the same as 0x7ff7b6a50568
//0x7ff7b6a5056c is the same as 0x7ff7b6a5056c
//The base address of arr is: 0x7ff7b6a50560

int main() {
  int a = 5;
  std::cout << "The address of a is: " << &a << std::endl;

  int arr[] = {1, 2, 3, 4};
  int size = sizeof(arr) / sizeof(arr[0]);

  // An integer typically takes 4 bytes. Note that pointer arithmetic
  // (arr + i) automatically scales by the element size, so it correctly
  // points to the i-th element.
  for (int i = 0; i < size; i++) {
    std::cout << &arr[i] << " is the same as " << (arr + i) << std::endl;
  }

  // In C++, an array name acts as a pointer to its first element.
  // 'arr' is equivalent to '&arr[0]' or 'arr + 0'.
  std::cout << "The base address of arr is: " << arr << std::endl;

  return 0;
}
```

2 Why Use a Pointer?

Answer: there are at least two usages of pointer in C++.

- Use pointer to allocate dynamically allocated memory. So we can have an array with a variable size.
- Pointers are necessary for implementing polymorphism, an important concept in object-oriented programming (later)

3 What are Pointers?

1. A pointer variable is defined as follows.

```
Type* pointerVariable;
```

2. The value of a pointer variable is the address of a variable of the same type. For example,
   - An int-type pointer variable saves the address of an `int` variable.
   - A double-type pointer variable saves the address of a `double` variable.
   - A string-type pointer variable saves the address of a `string` variable.

3. To find out the address of a variable, use `&` followed by the variable name. Here is an example.

```
int i = 5;
int* pInt = &i;       // pInt is an int pointer

double d = 6.5;
double* pDouble = &d; // pDouble is a double pointer

std::string str("Hello");
std::string* pStr = &str; // pStr is a string pointer
```

3.1 Dereferencing Pointers

1. To access or modify the value stored at the address a pointer holds, use the **dereference operator** (`*`).

2. When you dereference a pointer, you follow the address to the actual data.

```
int i = 5;
int* pInt = &i;

// Accessing the value
int value = *pInt;  // value is now 5

// Modifying the value through the pointer
*pInt = 10;         // i is now 10
```

3. Note that the `*` symbol has two different meanings depending on context:
   - In a **declaration** (`int* p`), it identifies the variable as a pointer.
   - In an **expression** (`*p = 10`), it acts as the dereference operator.

3.2 Safety: Null and Uninitialized Pointers

1. A pointer that is not initialized contains a **garbage address**. Attempting to dereference it will likely cause a program crash (Segmentation Fault).

2. Always initialize a pointer. If you do not have an address yet, set it to `nullptr`.

```
int* p = nullptr;  // Safe initialization
```

3. Before dereferencing, check if the pointer is null:

```
if (p != nullptr) {
    std::cout << *p;
} else {
    std::cout << "Pointer is null!";
}
```

4. **Warning:** Never dereference a `nullptr`. This is a common source of undefined behavior and application crashes.

4 What is the Difference Between `int* p` and `int *p`?

There is no difference. All of the following declarations are equivalent—each declares `p` as a pointer to an integer:

```
int* p;  // preferred style
int * p; // also valid
int *p;  // also valid
int*p;   // also valid (but avoid: hard to read)
```

5 What is a String Pointer?

A string pointer is a variable that stores the memory address of a `std::string` object. To access the string object it points to, use the dereference operator (`*`). When a string pointer is initialized to `nullptr`, it does not point to any valid object:

```
std::string* pStr = nullptr; // pStr does not point to any object
```

Or it can point to an existing string object using the address-of operator (`&`):

```
std::string str("Hello");
std::string* pStr2 = &str; // pStr2 holds the address of str
```

Here, `pStr2` and `str` are two ways to access the same object. Dereferencing the pointer (`*pStr2`) yields the actual string value.

6 What are the Common Uses for Two Pointers (Answer to the Lecture Slip)?

One usage of two pointers is to exchange the variables whose addresses are saved in those pointers. Here is an example to sort two strings by their lengths.

Code link: `https://onlinegdb.com/SDCsYpgtH`.

```
// Purpose: given two string pointer variables, if the string pointed to by
// the first pointer has more characters than that of the string pointed to
// by the second pointer, swap those two strings.
// Sample output:
```

```
//   after calling sort2(string*, string*) function
//   s1 = Hi
//   s2 = Hello
#include <iostream>
#include <string>
#include <utility> // std::swap is in the utility library since C++11

void sort2(std::string* ps1, std::string* ps2);

int main() {
  std::string s1("Hello");
  std::string s2("Hi");

  sort2(&s1, &s2);
  // &s1 is the address of string s1, passed to a string pointer parameter.
  // So is &s2.

  std::cout << "after calling sort2(string*, string*) function" << std::endl;
  std::cout << "s1 = " << s1 << '\n';
  std::cout << "s2 = " << s2 << '\n';
  return 0;
}

void sort2(std::string* ps1, std::string* ps2) {
  // (1) The type of ps1 is string*, so ps1 is a pointer to a string.
  // (2) Apply the dereference operator * to a pointer variable to access
  //     the variable whose address is saved in that pointer,
  //     that is, the object the pointer points to.
  //     For example, *ps1 is the string variable whose address is saved in ps1.
  //     In short, *ps1 is a string.
  // (3) (*ps1).length() calls the length() method on string variable *ps1
  //     and returns its number of characters.
  //     (*ps1).length() can be rewritten as ps1->length().
  // (4) Warning: do not omit the parentheses around *ps1 in (*ps1).length().
  //     The dot operator (.) has higher precedence than *,
  //     so *ps1.length() is parsed as *(ps1.length()), which fails because:
  //     (4a) ps1 is of type string*, a pointer -- it has no length() method.
  //     (4b) Even if ps1.length() could run, it returns an integer,
  //          and the dereference operator * requires a pointer, not an integer.
  if (ps1->length() > ps2->length()) {
    std::swap(*ps1, *ps2);
  }
}
```

7 Integer Array Passes as a Pointer to Integer

Arrays pass a pointer to the first element, I heard this is an alternate way to initialize an array as "`<type>* <name>`". How is size defined when declaring arrays with pointer notation?

Answer: You are right.

1. The name of an array also implies the address of the first element in the array. To save address of an element, we can use a pointer to that element's type. So, to declare an integer array in a function header, either one of the following is fine.

```
Type arrayName[];
//use the notation of array
```

```
Type* arrayName;
//use the notation of pointer}
```

2. When defining a function with an array, knowing the address of the first element of the array is not enough, we still need to know the size of the array, that is, the number of elements in the array. In short, when working with array, two things are needed.

   (a) Array name, which implies the address of the first element of the array. This information tells us where to find the first element of the array.

   (b) The size of the array. This information let us know when to stop working with elements in the array.

3. For example, to print the elements of an integer array, we can use either one of the following.

```
void print(int arr[], int  size);
```

```
void print(int* arr, int size);
```

4. In a function to work with an array, if size is not passed as a parameter, error would happen, as shown in the following example.

```
//Error message when the size of an array is not passed as a parameter to a
    function working with an array.

//warning: sizeof on array function parameter will return size of 'int *' instead
    of 'int[]' [-Wsizeof-array-argument]
//   15 |   int size = sizeof(arr) / sizeof(arr[0]); //WRONG
//      |                     ^
#include <iostream>
#include <string>

void print(int arr[]);

int main() {
  int arr[] = {1, 2, 3, 5};
  print(arr);

  return 0;
}

void print(int arr[]) {
  int size = sizeof(arr) / sizeof(arr[0]); //WRONG
  for (int i = 0; i < size; i++) {
```

```
    std::cout << arr[i] << std::endl;
  }
}
```

8 What do `*variable` and `&variable` mean?

The dereference operator `*` is followed by a pointer variable whose value is the address of the variable it points to. The address-of operator `&` is followed by a variable name and returns the address of that variable. Here is an example using a string pointer.

```
std::string* pStr;            // pStr is a string pointer variable
std::string str("Hello");     // str is a string variable

pStr = &str;  // store the address of str in pStr

// Note: &pStr is the address of the pointer variable pStr itself,
//       since pStr is also a variable and occupies a memory location.

// Note: *str is wrong -- str is not a pointer variable.

std::cout << *pStr << std::endl;
// *pStr dereferences pStr and returns the value of the string it points to.
// In this example, *pStr returns the value of str, which is "Hello".
```

9 Why use `nullptr` instead of `NULL` or 0?

In modern C++, `nullptr` is preferred because it is strongly typed. While `0` and `NULL` can be mistaken for integers, `nullptr` specifically represents a null pointer, preventing accidental bugs in overloaded functions.

10 How to Delete a Pointer?

Technically, we delete the memory the pointer points to, not the pointer variable itself—so "deallocate" is the more precise term. When memory is allocated dynamically via `new`, you must use `delete` to free it once it is no longer needed:

```
std::string* pStr = new std::string("Hello"); // dynamically allocated
// ... use the pointer ...
delete pStr; // frees the allocated memory
pStr = nullptr; // reset to avoid a dangling pointer
```

Only use `delete` on pointers allocated with `new`. Deleting a pointer to a local (stack) variable causes undefined behavior. For example, the following is **incorrect**:

```
std::string str("Hello");
std::string* pStr = &str;
delete pStr; // WRONG.
// 'str' is a local variable whose address
// is automatically allocated on the stack.
```

```
// Only dynamically allocated memory (via 'new')
// can be deleted.
```

We will discuss dynamically allocated memory in more detail in the next lecture.

11 When to Use * (Pass by Pointer) and When to Use & (Pass by Reference) ?

Answer:

1. Pass by reference and pass by pointer are different ways to access the original data and may modify the data if necessary.

   (a) Pass by reference example: to stamp a visa on a passport, need to pass the original copy of passport.

   (b) Pass by pointer (also called pass by address) example: a social worker needs to evaluate a kid. Given the kid's address, the social worker can visit the kid.

   (c) As another example, consider three ways to interact with a person:

      i. **Pass by Value:** A clone of the person is sent. Changes made to the clone do not affect the original unless the modified copy is explicitly returned. This cloning process also incurs significant overhead.

      ii. **Pass by Reference:** You work directly with the actual person. Any changes are permanent, and no copying is required. This is often a simpler and safer option, since you are guaranteed a valid person to interact with.

      iii. **Pass by Address (Pointer):** A copy of the person's address is sent. Although only the address is copied, you can still visit the person at that address and interact with them directly—producing the same effect as pass-by-reference. However, if the address is invalid (i.e., the pointer is `nullptr`), there is no one to interact with. Due to this added complexity and the risk of invalid addresses, pass-by-reference is generally preferred when direct access to the original is needed.

   Here is a table to summarize the above explanation:

Method	**What is Sent**	**What You Use**	**Effect of Changes**	**Notes**
Pass by Value	A clone	The clone	Original unchanged*	*Unless returned; high copying overhead
Pass by Reference	Direct access	The actual person	Affects original	Simple and reliable (always valid)
Pass by Address	An address copy	The person via address	Affects original	Pass-by-value of address; may be `nullptr`
Pass by Const Ref	Direct access	The actual person	**Read-only**	No copying; prevents accidental changes

2. When we need to pass the address of a variable to a function, use Type*, where Type is the type of that variable.

3. One usage of passing by address is to exchange two variables. In that case, we use the address to find out the elements, and exchange them. One such example is in Section 6.

4. Another usage of passing by address is to work with an array, we need to pass the array name and its size (number of elements in this array). Note that the name of the array also implies the initial address of the array. So the formal parameter of the function needs to be Type* parameter.

   For example, use int* to hold (aka save) the initial address of the array of integers.

```
void print(int* arr, int size);

//the above function header is same as
//void print(int arr[], int  size);
```

   The following is a better option if you do not plan to change the contents of the array.

```
void print(const int* arr, int size);

//the above function header is same as
//void print(const int arr[], int  size);
```

5. When we need to change an actual parameter of a function and want to carry that change back to a caller of that function. However, that value cannot be returned due to return type is used by another argument (there is at most one return in a function) or the return type is void. Under this situatation, use `Type&` in function header.

   For example, when we add an element to the an array, the size (current number of elements) of an array may be changed. If we do not want to return the size to a caller of the function and set return type as void, we pass that parameter by reference and use Type&.

```
void append(int* arr, int capacity, int& size, int toInsert);
//arr: name of array, which also implies the initial address of the array
//capacity: the maximum number of elements to be stored in that array
//size: the number of elements currently stored in the array
//toInsert: the element to be insert to the array

//The above function header can also be written as
//void append(int arr[], int capacity, int& size, int toInsert);
```

6. Please read the following links for further information.

   `https://stackoverflow.com/questions/7058339/when-should-i-use-pointers-instead-of-referen`

   `https://www.geeksforgeeks.org/cpp/passing-by-pointer-vs-passing-by-reference-in-cpp/`

   The following table and example comes from the link in `geeksforgeek`.

Parameters	Pass by Pointer	Pass by Reference
Passing Arguments	We pass the address of arguments in the function call.	We pass the arguments in the function call.
Accessing Values	The value of the arguments is accessed via the dereferencing operator *	The reference name can be used to implicitly reference a value.
Reassignment	Passed parameters can be moved/reassigned to a different memory location.	Parameters can't be moved/reassigned to another memory address.
Allowed Values	Pointers can contain a NULL value, so a passed argument may point to a NULL or even a garbage value.	References cannot contain a NULL value, so it is guaranteed to have some value.

7. Difference Between Reference Variable and Pointer Variable

   (a) A reference is the same object, just with a different name and a reference must refer to an object. Since references can't be NULL, they are safer to use.

   (b) A pointer can be re-assigned while a reference cannot, and must be assigned at initialization only.

   (c) The pointer can be assigned NULL directly, whereas the reference cannot.

   (d) Pointers can iterate over an array, we can use increment/decrement operators to go to the next/previous item that a pointer is pointing to.

   (e) A pointer is a variable that holds a memory address. A reference has the same memory address as the item it references.

   (f) A pointer to a class/struct uses `->` (arrow operator) to access its members whereas a reference uses a '.' (dot operator)

   Code to illustrate the above ideas is shown in `https://onlinegdb.com/KlH3Ei2XeH`.

```cpp
// C++ program to demonstrate differences
// between pointer and reference
//code from https://www.geeksforgeeks.org/cpp/passing-by-pointer-vs-passing-by-reference-in-cpp/
//Sample input/output:
//0x7fff69ec8cc0 0x7fff69ec8cd4
//0x7fff69ec8cd4 0x7fff69ec8cd4
//0x4
//7

#include <iostream>

struct demo {
    int a;
};

int main() {
    int x = 5;
    int y = 6;
```

```
    demo d;

    int* p;
    p = &x;
    p = &y; // 1. Pointer reinitialization allowed

    int& r = x;
    // &r = y;                  // 1. Compile Error

    r = y; // 1. x value becomes 6

    p = NULL;
    // &r = NULL;              // 2. Compile Error

    // 3. Points to next memory location
    p++;

    // 3. x values becomes 7
    r++;

    std::cout << &p << " " << &x << '\n'; // 4. Different address
    std::cout << &r << " " << &x << '\n'; // 4. Same address

    demo* q = &d;
    demo& qq = d;

    q->a = 8;
    // q.a = 8;                  // 5. Compile Error
    qq.a = 8;
    // qq->a = 8;              // 5. Compile Error

    // 6. Prints the address
    std::cout << p << '\n';

    // 6. Print the value of x
    std::cout << r << '\n';

    return 0;
}
```

(g) A pointer needs to be dereferenced with * to access the memory location it points to, whereas a reference can be used directly.

8. Use references wherever you **can**, and pointers wherever you **must**.

   Avoid pointers until you can't.

   The reason is that pointers make things harder to follow/read, less safe and far more dangerous manipulations than any other constructs.

So the rule of thumb is to use pointers only if there is no other choice.

12 When to Use Const for a Function

How do I know when to use const for a function?

Answer: the following explanation and code comes from google when searching for "When to Use Const for a Function".

For parameters passed by reference or pointer when the function does not modify them:

If a function receives a parameter by reference (&) or pointer (*) but does not intend to modify the data it points to or refers to, declaring it const is crucial. This prevents accidental modification and improves code safety.

```
void printData(const std::string& data) {
  // const reference parameter
  // data = "new data"; // Invalid: cannot modify const reference
  std::cout << data << std::endl;
}
```

Another example is shown as follows. Note: since this is a first course in C++, for simplicity, sometimes we omit const in the function header of declaration and definition.

```
//purpose: illustrate const int* in function parameter to print an integer array.
//sample output:
//1 2 3 5
#include <iostream>
#include <string>

void print(const int* arr, int size);

int main() {
  int arr[] = {1, 2, 3, 5};
  int size = sizeof(arr) / sizeof(arr[0]);
  print(arr, size);

  return 0;
}

//void print(const int arr[], int size) { //also ok
void print(const int* arr, int size) {
  for (int i = 0; i < size; i++) {
    std::cout << arr[i] << ' ';
  }
  std::cout << std::endl;

  //arr[0] = 3; //error: read-only variable is not assignable
}
```

To convey intent and improve readability:

Using const for parameters clearly signals to other developers that the function will not alter the passed-in arguments, making the code easier to understand and maintain.

In summary:

Employ const for member functions that do not alter the object's state, and for function parameters passed by reference or pointer that are not intended for modification. This practice enforces const correctness, enhances code safety, and improves clarity.

13 Did I use the right steps to get the string pointer?

Answer: Here is your code.

```
#include <iostream>
#include <string>
using namespace std;

string sort2() {
    string one;
    cout << "Enter string: ";
    cin >> one;
    string p1 = &one;   // wrong

    string two;
    cout << "Enter next string: ";
    cin >> two;
    string p2 = &two;   // wrong
}
```

The input parts are correct.

The following statement to declare and initialize string pointer variable `p1` is wrong. Variable `p1` is declared as a `string` variable, whose value is a sequence of characters enclosed in a pair of matched double quotes. A `string` variable **cannot** save the address of string variable `one`.

```
    string p1 = &one;   // wrong
```

Here is a fix.

```
    string* p1 = &one;  // correct
```

Similarly, the following statement is wrong, and should be fixed in the same way.

```
    string p2 = &two;   // wrong
```

Here is the fix.

```
    string* p2 = &two;  // correct
```

14 I still don't fully understand pointers

Pointers are a notoriously challenging topic in C++. If you don't fully understand them yet, don't worry—mastery comes with practice. Be patient and persistent; with enough repetition, the concept will eventually "click."

15 Will Monte Carlo pi simulation be on the final?

Answer: The Monte Carlo method for estimating π will not be on the final exam. However, generating a random integer or floating-point number within a specific range will be included.

16 Not Enough Time for the Lecture Slip Question

Q: Why is the lecture slip question taught in the last 5 minutes of class?

Q: Can you please start the slides at the beginning of the class? The last 10 minutes are not enough to learn the topics.

Q: Could we begin working on the lecture slip earlier?

Answer: I want to focus on explaining the reasoning process so that students understand how to think through problems, rather than simply memorizing solutions. However, I may have spent too much time on a single topic.

Moving forward, I will aim to introduce new topics earlier and allocate more time to working through lecture slip problems.

17 I Need the Code Posted

Answer: source codes discussed in class will be posted in bright space.